INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

UNIDAD DE APRENDIZAJE:
ANÁLISIS Y DESARROLLO DE ALGORITMOS

PRÁCTICA 2
COMPLEJIDAD EN ORDENAMIENTO

Implementación y evaluación en Java y Python

Profesor: ANDRES GARCIA FLORIANO

Alumno:
Hernandez Barrios Samuel Rodrigo

Tercer semestre

3BV1

Fecha de entrega: 27 de octubre de 2026

1 Objetivo y método experimental

Se implementan Bubble Sort, Merge Sort, Tree Sort, Heap Sort y Counting Sort en Java y Python. Se mide tiempo de ordenamiento y pico de memoria del proceso con datos reproducibles; se valida cada resultado y se contrasta el comportamiento observado con la complejidad teórica.

Datos de entrada y adaptación de Counting Sort

Se usan arreglos de números reales representados en float de Python y double de Java, dentro de [−500.00, 500.00] en pasos de 0.01. Son valores discretizados: se generan claves enteras y se dividen entre 100. Counting Sort trabaja con las claves en centésimas; no se afirma que ordene directamente cualquier real arbitrario. Entradas con más de dos decimales se rechazan; NaN e infinitos quedan fuera del dominio.

Ambos lenguajes reciben la misma secuencia: estado inicial 2026; x siguiente = (1664525x + 1013904223) módulo 2^32; clave = (x módulo 100001) − 50000. El rango fijo produce repeticiones y favorece los métodos sensibles al número de claves distintas.

Protocolo

Cada caso completado tiene tres repeticiones, ejecutadas secuencialmente en procesos nuevos. El cronómetro incluye solo el algoritmo: excluye generación, calentamiento y validación. Se usa perf_counter en Python y nanoTime en Java. Se ordenan 200 elementos antes de medir: una vez en Python y cinco en Java. El calentamiento es breve y no elimina todos los efectos del JIT y de la recolección de basura.

Se reporta la mediana de los tres tiempos y el máximo de los picos de memoria. PeakWorkingSetSize se consulta desde el proceso controlador cada 20 ms. Mide memoria residente del proceso completo, incluidos runtime, entrada, contadores de validación y algoritmo; no equivale al espacio auxiliar teórico. MiB = 2^20 bytes.

Validación y controles

Las diez implementaciones pasaron 70 pruebas pequeñas en total, incluyendo vacío, negativos, duplicados, orden inverso y datos generados. Se compararon contra el ordenamiento de biblioteca, utilizado solo como referencia de prueba. En cada ejecución grande se comprueban orden no decreciente y conservación exacta del histograma de las 100001 claves.

Los límites son 8 s de vida de proceso para Bubble y 45 s para los demás; un corte no se registra como tiempo completo de ordenamiento. Se limita el pico del proceso a 256 MiB y la JVM a un heap de 256 MiB. Una preselección conservadora evita iniciar casos con demanda prevista superior al presupuesto o a la RAM libre menos 128 MiB.

2 Implementaciones y complejidad

Algoritmo	Implementación	Tiempo	Auxiliar
Bubble	Intercambios adyacentes y salida temprana	Mejor Θ(n); promedio y peor Θ(n²)	Θ(1)
Merge	Mezcla iterativa por anchos 1, 2, 4...	Θ(n log n)	Θ(n)
Tree	BST sin balance; duplicados contados; recorrido iterativo	Esperado O(n log(u+1)); peor O(nu)	O(u+h)
Heap	Max heap; extracción del máximo	O(n log n); peor Θ(n log n)	Θ(1)
Counting	Frecuencias y reconstrucción de centésimas	Θ(n+k)	Θ(k)

n es el número de elementos; u, el número de valores distintos; h, la altura del BST; k, el ancho del rango de claves. En esta prueba k ≤ 100001 y u ≤ 100001. El análisis esperado de Tree supone una distribución favorable de inserción; no garantiza balance. Con valores distintos ordenados, u = n y el peor caso es Θ(n²).

Decisiones de implementación

Merge utiliza un arreglo auxiliar y alterna entre los dos buffers; el llamador conserva el arreglo devuelto. Heap y Bubble reorganizan el arreglo original. Tree conserva un nodo por clave distinta y escribe el recorrido en el arreglo. Counting reconstruye las cifras a partir de frecuencias, sin mantener un segundo arreglo de n posiciones.

El uso de un contador de duplicados reduce los nodos de Tree en esta distribución. Counting tiene un rango pequeño y fijo; por ello su ventaja en estos datos no debe generalizarse a reales sin discretización o a intervalos enormes. Merge y Bubble conservan el orden relativo de iguales. Heap no es estable; Counting reconstruye valores y no promete estabilidad para registros con identidad.

Entorno de ejecución

Sistema: Windows-11-10.0.26200-SP0. CPU informada: Intel64 Family 6 Model 186 Stepping 2, GenuineIntel. RAM física: 15.67 GiB; disponible al iniciar: 1249.1 MiB. Las versiones y opciones completas están guardadas en los archivos entorno de resultados.

Python: 3.12.14 (main, Aug 25 2026, 14:01:42) [MSC v.1944 64 bit (AMD64)]. Java: java version "26.0.2.1" 2026-08-18.

Las mediciones corresponden a esta computadora y a estas implementaciones. La carga de otras aplicaciones, la presión de memoria, las cachés y el JIT pueden afectar los valores. No se interpretan como una comparación universal de velocidad entre lenguajes.

3 Resultados con tamaños solicitados

Cada fila completada resume tres ejecuciones correctas. Los tiempos son medianas en segundos; la memoria es el máximo del pico residente del proceso en MiB. No se mezclan memoria medida y estimaciones teóricas.

Tamaño de 100,000 elementos

Algoritmo	Python s	Java s	Python MiB	Java MiB
bubble	No completado	No completado	—	—
merge	0.125146	0.011918	19.12	44.98
tree	0.128223	0.015861	21.59	46.86
heap	0.224513	0.009607	17.61	43.39
counting	0.026817	0.001810	18.44	46.63

Tamaño de 1,000,000 elementos

Algoritmo	Python s	Java s	Python MiB	Java MiB
bubble	No completado	No completado	—	—
merge	1.687093	0.090495	60.21	59.78
tree	1.376046	0.142293	58.70	54.95
heap	6.558417	0.121171	57.82	51.18
counting	0.215202	0.011712	57.26	53.88

Bubble con 100000 elementos fue interrumpido en ambos lenguajes al alcanzar 8 s de vida del proceso. No hay un tiempo completo que se pueda reportar. Con un millón y tamaños mayores se omitió por presupuesto de tiempo. El CSV conserva el pico observado durante la ejecución interrumpida, pero no se incluye como una medición completa en estas tablas.

4 Ampliación y pruebas de Bubble Sort

Diez millones de elementos en Java

Algoritmo	Mediana s	Pico máximo MiB	Repeticiones
merge	0.961376	199.07	3
tree	1.445966	125.47	3
heap	1.972883	121.56	3
counting	0.116087	122.59	3

Los cuatro algoritmos completaron tres ejecuciones correctas en Java. En Python este tamaño no se inició porque excedía el presupuesto conservador de memoria de la sesión. Los casos de cien millones y mil millones tampoco se iniciaron en ninguno de los dos lenguajes por ese presupuesto; Bubble tiene además la restricción de tiempo.

Bubble Sort con tamaños complementarios

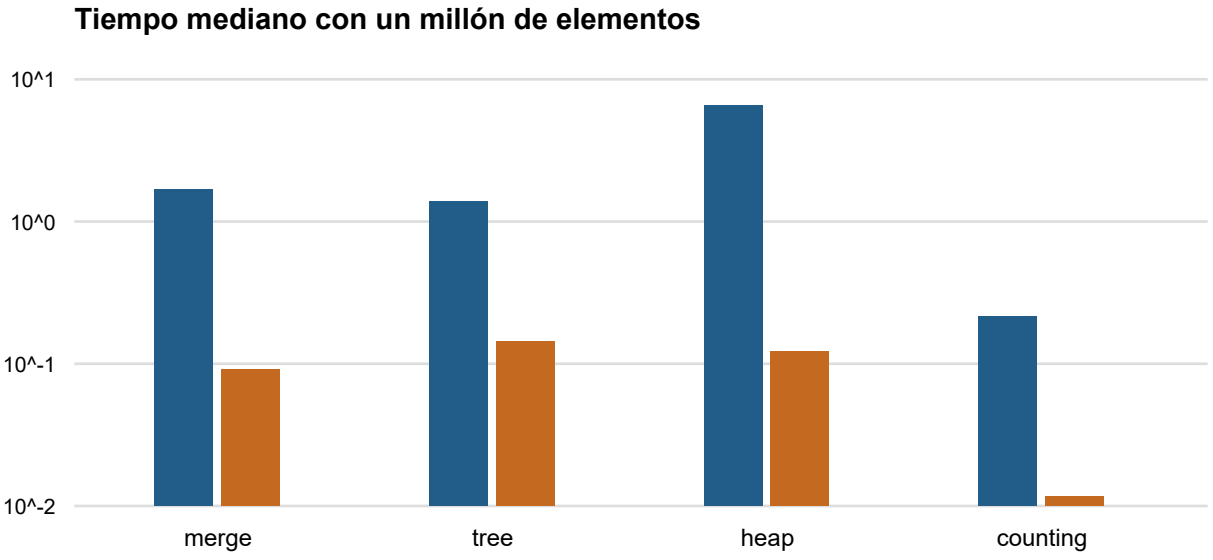
Elementos	Python s	Java s	Python MiB	Java MiB
1,000	0.025522	0.001443	13.22	41.85
5,000	0.693677	0.011951	13.39	42.41
10,000	2.926752	0.066701	13.64	42.52

Estas pruebas complementarias permiten observar Bubble sin esperar una ejecución enorme. No reemplazan las longitudes solicitadas. En Python, duplicar el tamaño de 5000 a 10000 aumenta el tiempo aproximadamente 4.22 veces, coherente con un crecimiento cuadrático; no constituye una demostración a partir de una sola distribución.

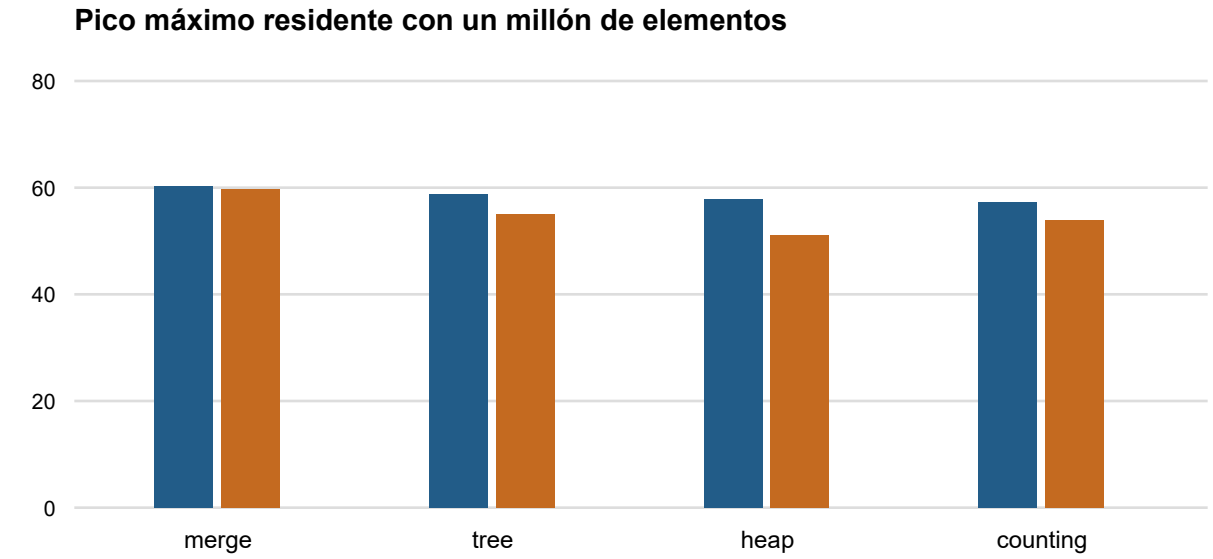
Estado de cobertura

Se implementaron los cinco métodos en ambos lenguajes. Hay resultados completos para cuatro métodos a 100000 y un millón en ambos, y a diez millones en Java. Las longitudes mayores y Bubble desde 100000 no tienen mediciones completas. Por tanto, la ejecución experimental cubre parcialmente las longitudes de la consigna; completar todos los casos requiere otros recursos o acordar límites con el profesor.

5 Gráficas e interpretación



Azul: Python Naranja: Java | Eje vertical logarítmico en segundos



Azul: Python Naranja: Java | Memoria del proceso completo en MiB

Counting fue el más rápido entre los métodos completados para esta distribución. Su trabajo depende de n y de un rango de 100001 claves. En Java, con diez millones de elementos, el orden temporal observado es Counting, Merge, Tree y Heap. Estos resultados se refieren a la secuencia y al rango empleados.

La memoria total de Java incluye la JVM; la de Python incluye el intérprete y los objetos float. Un pico de proceso mayor no implica por sí solo una complejidad auxiliar mayor. La escala logarítmica de la gráfica de tiempo permite comparar órdenes de magnitud sin ocultar los valores más pequeños.

6 Viabilidad de las longitudes mayores

La siguiente tabla es un cálculo teórico, no una medición. Solo cuenta almacenamiento de valores de 8 bytes en Java y, para Merge, dos buffers. Excluye runtime, contadores, nodos, encabezados y memoria temporal. En Python una lista de float requiere además referencias y objetos; no basta con multiplicar n por ocho.

Elementos	Un arreglo de 8n bytes	Dos buffers de 16n bytes	Comparaciones Bubble máximas
100,000	0.0007 GiB	0.0015 GiB	4,999,950,000
1,000,000	0.0075 GiB	0.0149 GiB	499,999,500,000
10,000,000	0.0745 GiB	0.1490 GiB	49,999,995,000,000
100,000,000	0.7451 GiB	1.4901 GiB	4,999,999,950,000,000
1,000,000,000	7.4506 GiB	14.9012 GiB	499,999,999,500,000,000

Para mil millones de valores, el arreglo básico requiere 7.45 GiB y dos buffers 14.90 GiB. Esa cantidad supera ampliamente la RAM libre observada y no deja espacio suficiente para el sistema ni para el runtime. La ausencia de resultados en esos tamaños queda documentada como no ejecutado, nunca como cero segundos ni como una ejecución exitosa.

En Bubble, $n(n - 1)/2$ es el máximo de comparaciones de los ciclos decrecientes de esta implementación, alcanzado cuando no hay salida temprana. Es un conteo de operaciones, no una extrapolación de segundos. No se asignó tiempo estimado a las longitudes no terminadas.

Cómo continuar en otro entorno

El ejecutor incluye las cinco longitudes, registro de errores, límites configurables y exportación CSV. Debe ajustarse el presupuesto al equipo y, en Java, también el parámetro -Xmx del comando. Para tamaños grandes conviene una máquina dedicada y una política acordada para los métodos cuadráticos. No basta con aumentar el límite de memoria sin verificar RAM disponible.

El conjunto de números se genera dentro del proceso y los procesos se ejecutan secuencialmente; no se lanzan varios arreglos grandes en paralelo. Cada interrupción o preselección queda registrada con su causa en los datos crudos.

7 Reproducibilidad y conclusiones

Archivos del proyecto

python/ordenamientos.py contiene las cinco implementaciones y el generador; python/pruebas.py ejecuta las pruebas; python/caso.py mide y valida un caso. java/Ordenamientos.java contiene los algoritmos, pruebas y medición equivalentes. medir.py controla procesos y memoria. resultados contiene mediciones, resumen y entorno. README.md explica comandos, supuestos y límites.

Comandos desde la carpeta del proyecto:

```
python python/pruebas.py
javac java/Ordenamientos.java
java -cp java Ordenamientos test
python medir.py --java java --max-n 1000000000
```

Repositorio público de código y resultados: https://github.com/Samuel-Hdzx07/Practicas_ADA_3BV1. El paquete ZIP conserva las carpetas y los comandos reproducibles del proyecto.

Conclusiones

La complejidad teórica explica por qué Bubble se vuelve impráctico antes que los métodos n log n. Counting muestra una ventaja clara cuando se restringe el dominio de claves. Tree requiere considerar altura y duplicados; una entrada favorable no elimina su peor caso cuadrático. La comparación de tiempos solo tiene sentido junto con el dominio, las implementaciones y el protocolo utilizado.

Las mediciones completas fueron verificadas y se conservaron sus repeticiones. Los casos omitidos e interrumpidos son parte del resultado experimental y señalan los límites del equipo y de la configuración; no completan por sí mismos la exigencia de ejecutar todas las longitudes.

Referencias y declaración de apoyo de IA

OpenAI. (2026). ChatGPT [Herramienta de inteligencia artificial generativa]. https://chatgpt.com.

Se utilizó ChatGPT como apoyo en el formato, la organización y el contenido.